

Ha & Schmidhuber (2018) - World Models

Literature Review

Soo Min Kimm

08/10/26

- 1 World, Agent, and World Model
- 2 Ha & Schmidhuber (2018)
- 3 Conclusion

Outline

- 1 World, Agent, and World Model
- 2 Ha & Schmidhuber (2018)
- 3 Conclusion

World and Agent

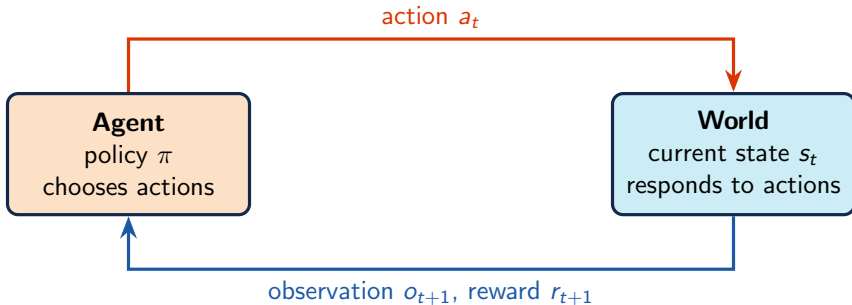
World

The system whose changes matter for the task: its entities, their current state, and how they evolve under actions. In robotics, this includes the robot's body and its surroundings.

Agent

The decision-making system that receives observations and chooses actions to pursue a goal. Its rule for choosing actions is called a **policy**.

The Agent–World Loop



- Observation o_t is what the agent senses about the world state s_t .
- In RL, reward r_{t+1} gives feedback about the task after action a_t .

The Agent–World Loop as a POMDP

The agent–world loop can be described as a partially observable Markov decision process (POMDP):

States s_t	The world's physical situation, including the robot's body
Actions a_t	Commands chosen by the agent
Transition $T(s_{t+1} \mid s_t, a_t)$	Distribution of the next state after an action
Observation $O(o_t \mid s_t)$	Distribution of what the agent senses in a state
Reward $R(s_t, a_t)$	Expected immediate task feedback

The agent receives observations rather than direct access to the full state. The state evolves under actions; the agent senses it through observations.

What a World Model is

World model

Given the history of observations and actions

$$H_t = (o_0, a_0, o_1, a_1, \dots, a_{t-1}, o_t)$$

and a candidate action a_t , a **world model** predicts a probability distribution over the next observation o_{t+1} :

$$p_{\theta}(o_{t+1} \mid H_t, a_t)$$

where θ denotes the model's learned parameters.

- **Policy:** “What should I do?”
- **World model:** “What would happen if I did this?”

What a World Model is *NOT*

Component	Predicts	Policy-dependent?	Belongs to
Dynamics $p(s_{t+1} s_t, a_t)$	next state	no	world model
Reward model $r(s_t, a_t)$	immediate reward	no	world model (optional)
Value $V^\pi(s) = \mathbb{E}_\pi[\sum_k \gamma^k r_{t+k}]$	future return	yes	agent (critic)
Policy $\pi(a_t s_t)$	action	–	agent (actor)

Outline

1 World, Agent, and World Model

2 Ha & Schmidhuber (2018)

- Architecture Overview
- Vision (V)
- Memory (M)
- Controller (C)
- Training Procedure
- Car Racing
- VizDoom
- Lessons and Limitations

3 Conclusion

The V-M-C Architecture

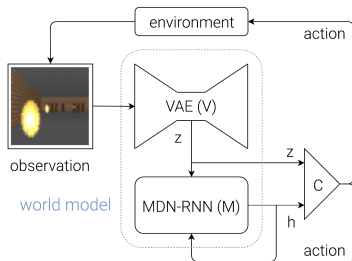


Figure 8. The V-M-C agent.

Source: Ha & Schmidhuber (2018), Section 2.4.

- **V: vision.** Compress image o_t into a latent vector z_t .
- **M: memory and prediction.** Predict the next latent vector from z_t , action a_t , and memory h_t .
- **C: controller / policy.** Choose action a_t from the current latent vector and memory.

Agent = V + M + C.

World model = V + M.

An internal learned model of the environment.

Design principle: learn a large predictive representation; optimise a small controller for reward.

V: Vision with a Convolutional VAE

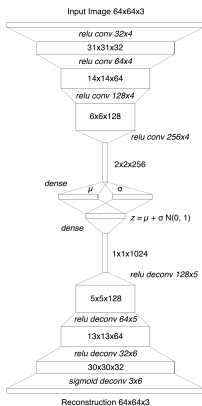


Figure 22. The complete ConvVAE.
Source: Ha & Schmidhuber (2018),
Appendix A.1.

Architecture

- Encoder: four convolutional layers compress a 64×64 RGB image.
- Sample from the encoder's Gaussian:

$$z_t = \mu_t + \sigma_t \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

- Decoder: four transposed convolutions.

Training: reconstruction + KL; no reward.

Design choices

- Compact latent vector:** 32 values (Car Racing); 64 (VizDoom).
- Gaussian prior:** KL encourages encoder distributions towards $\mathcal{N}(0, I)$.

M: Memory with an MDN-RNN

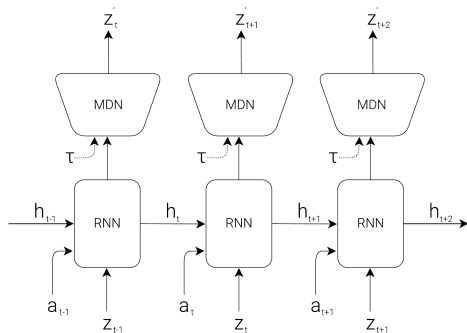


Figure 6. MDN-RNN unrolled over time.
Source: Ha & Schmidhuber (2018), Section 2.2;
Appendix A.2.

Architecture

- LSTM memory: 256 units (Car Racing); 512 (VizDoom).
- Predict each vector entry with five Gaussian components (factored output).
- VizDoom: additional termination head.

Training: make recorded next latent vectors likely (negative log-likelihood).

Design choices

- **Memory:** retain earlier information.
- **Uncertainty:** allow different futures.
- **Latent simulation:** avoid pixel rendering.

C: A Small Controller

Architecture: join the latent vector and memory, then take weighted sums plus a bias:

$$u_t = W_c[z_t; h_t] + b_c, \quad a_t = g_{\text{task}}(u_t).$$

g_{task} applies the task's action bounds and mapping.

Training: freeze V and M; CMA-ES optimises C using rollout return.

cf.CMA-ES: Covariance Matrix Adaptation Evolution Strategy.

Design choices

- **No hidden layer:** keep reward optimisation in a small parameter space.
- **Read z_t and h_t :** combine current perception with temporal predictive features.
- **Reward-only search:** sample controllers, rank by return, adapt the search distribution.

Why Train V, M, and C Separately?

Start with data: collect 10,000 random-policy rollouts for each environment.

Stage	Learning signal	What is learned
1. Train V	Reconstruction + KL	Compact visual representation
2. Train M	Recorded next latent vectors	Action-conditioned predictive memory
3. Optimise C	Rollout return, using CMA-ES	Behaviour from fixed V/M features

Design rationale: learn perception and dynamics from observation data; reserve reward optimisation for the small controller.

Two uses of the same design

- **Car Racing:** optimise C in the actual environment using V/M features.
- **VizDoom:** optimise C inside M's latent simulator, then test in the actual game.

Actual Environment vs. Dream

	Actual game	Dream: M's latent simulator
Current latent vector z_t	V encodes the current image o_t	M supplies a sampled latent vector
Next latent vector z_{t+1}	V encodes the game's next frame	M samples the next latent vector
M's role	Update predictive memory	Memory + latent transitions
C's inputs	z_t plus recurrent memory	z_t plus recurrent memory
V during control	Encoder processes each image	Encoder bypassed; decoder optional for viewing

Why bypass V? The dream already provides latent vectors in V's latent space; there is no rendered frame to encode.

The latent vector is still used. C reads the *current* z_t and memory; M's sampled z_{t+1} becomes the current latent vector at the next step.

Car Racing: Predictive Features for Control

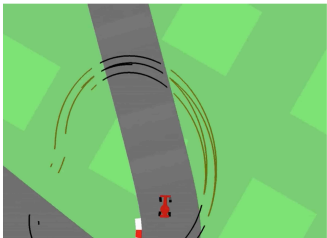


Figure 9. CarRacing-v0.

Source: Ha & Schmidhuber (2018),
Section 3; Table 1.

Features	Score
V only	632 ± 251
V + M	906 ± 21

100 random tracks; solve: ≥ 900 .

Task: navigate randomly generated tracks from images. Reward favours visiting track tiles while penalising elapsed time.

Actions: steering, acceleration, braking.

Design

- V: 32-dimensional latent vector; M: 256-unit LSTM.
- C maps $[z_t; h_t]$ to three bounded controls.
- Learn V/M from 10,000 random rollouts; then freeze them.
- CMA-ES optimises C in the **actual game**.

Role of M: predictive features for C, without an explicit planning search at each decision.

VizDoom: Learning Inside the Dream



Figure 14. VizDoom: Take Cover.
Source: Ha & Schmidhuber (2018),
Section 4; Table 2.

τ	Dream	Actual
0.10	2086 ± 140	193 ± 58
1.15	918 ± 546	1092 ± 556

Survival steps; actual test: 100 trials. Solve: > 750 .

Task: dodge monsters' fireballs and stay alive.

Actions: move left, stay, or move right.

Design

- V: 64-dimensional latent vector; M: 512-unit LSTM plus a termination head.
- C uses $[z_t; h_t; c_t]$; a scalar output maps to three choices.
- Learn V/M from 10,000 recorded random rollouts in the actual game.
- CMA-ES optimises C **only inside M's latent simulator**, then tests it in the actual game.

Design check: sampling temperature affects model exploitation and transfer.

Architectural Lessons and Limitations

- **Capacity allocation:** a large V/M representation supports a small reward-optimised controller.
- **V's bottleneck:** reconstruction retains visual information, which may differ from what control needs.
- **M's memory:** a finite LSTM state summarises history; simulated futures advance one step at a time.
- **Simulation errors:** C can exploit M; more stochastic dreams helped transfer but did not eliminate model bias.

Outline

- 1 World, Agent, and World Model
- 2 Ha & Schmidhuber (2018)
- 3 Conclusion**

Summary

- **V:** a convolutional VAE compresses each image into z_t .
- **M:** an LSTM with a mixture-density head predicts z_{t+1} from z_t , a_t , and memory.
- **C:** a small controller reads the current latent vector and recurrent features to choose actions.
- **Training:** learn V and M from recorded interactions, then optimise C for rollout return.
- **Evidence:** predictive features help Car Racing; latent-simulator training transfers in VizDoom, with sensitivity to model errors.

Comparison with Recent Models

Model	Prediction	Action selection	Main design difference
World Models (2018)	VAE latent vectors via MDN-LSTM	Small C, optimised by CMA-ES	Separate V/M/C; task-specific game data
V-JEPA 2-AC (2025)	Future JEPA features	MPC towards an image goal	Web-video pretraining, then robot dynamics
DreamZero (2026)	Future video and action chunks	Joint video–action diffusion policy	Future generation coupled to executable actions
GigaWorld-Policy (2026)	Actions + future video during training	Action output can skip video generation	Causal masking makes future video optional at inference

Connection: 2018 separates prediction from control; recent designs use prediction for latent planning or integrate it with action generation.

Thank you!

References

- K. Craik. *The Nature of Explanation*. Cambridge University Press, 1943.
- L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 1998.
- R. S. Sutton. Integrated architectures for learning, planning, and reacting (Dyna). *ICML*, 1990.
- R. S. Sutton and A. G. Barto. *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.
- D. Ha and J. Schmidhuber. World Models. *arXiv:1803.10122*, 2018.
- M. Assran et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*, 2025.
- N. Hansen. The CMA Evolution Strategy: A Tutorial. *arXiv:1604.00772*, 2016.
- D. Hafner et al. Learning latent dynamics for planning from pixels (PlaNet), *ICML 2019*; Dream to control (Dreamer), *ICLR 2020*.
- J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero). *Nature*, 2020.
- Y. LeCun. A path towards autonomous machine intelligence. *OpenReview*, 2022.
- J. Bruce et al. Genie: Generative interactive environments. *ICML*, 2024.
- A. Brohan et al. RT-2: Vision-language-action models. 2023. M. J. Kim et al. OpenVLA. 2024. K. Black et al. π_0 . 2024.
- Y. Du et al. Learning universal policies via text-guided video generation (UniPi). *NeurIPS*, 2023.
- Motus: A unified latent action world model. *arXiv:2512.13030*, 2025.
- S. Ye et al. World action models are zero-shot policies (DreamZero). *arXiv:2602.15922*, 2026.
- GigaAI. GigaWorld-Policy: An efficient action-centered world-action model. *arXiv:2603.17240*, 2026.
- X. Zhang, X. Zeng, and W. Zhang. From world models to world action models: A concise tutorial for robotics. *arXiv:2607.00836*, 2026.
- Z. Lu et al. World-action models for robot learning and control: A survey. *arXiv:2609.16074*, 2026.